

SEN381 | SOFTWARE ENGINEERING 381

Research Foundations

Assignment 1

PREPARED BY

Ryno Martin Goetz | 602306

Steven Riaan Piek | 602240

Willem Booysen | 600613

LECTURER

Ms E Shayamano

SUBMISSION DATE

3 September 2026

Contents

Question 1: Software Engineering Beyond Programming.....	3
Interaction between stakeholder needs and scope	3
Scope, schedule and cost.....	3
Schedule and resources	3
Cost and quality.....	4
Risk across all project factors	4
Real-world example: Case C	5
Question 2: Thinking Ahead - Decisions Across the Software Lifecycle.....	6
1. Requirements and scope.....	6
2. Methodology and process.....	6
3. Stakeholder alignment.....	6
4. Security and compliance.....	7
5. Integration	7
6. Maintenance.....	7
Decision horizon map	8
Question 3: Evidence-Based Technology Stack Decisions	9
3.1 Evaluation context and alternatives	9
3.2 Comparison against lifecycle criteria.....	9
3.3 Criteria weights and justification	11
3.4 Weighted decision matrix.....	11
3.5 Limitations, sensitivity and critical conclusion	12
Question 4: Generative AI in Software Engineering.....	13
4.1 Useful applications of generative AI in software engineering	13
4.2 Risks of AI-assisted engineering.....	13
4.3 Human verification and accountability.....	14
AI Research and Verification Record.....	15
References	16

Question 1: Software Engineering Beyond Programming

Software engineering goes beyond programming because stakeholder needs, scope, schedule, cost, quality and risk must be managed together in a software project. These factors are interdependent: Changing one can affect several others. Research on software project success shows that meeting scope / schedule / budget is not enough; satisfaction of stakeholders / requirements / product quality are other measures of success (Monserrat, Mas and Mesquida, 2025).

Interaction between stakeholder needs and scope

Software projects begin with stakeholder needs and expectations translated into software requirements. Requirements engineering is the process of eliciting, documenting and maintaining stakeholder requirements while requirements prioritisation determines which requirements are most important when not all requirements can be delivered at once (Hujainah et al., 2018). This puts stakeholder involvement in relation to scope because stakeholder priorities determine what features/functions should be included in the project. Hujainah et al (2018) found organisations often have limited human expertise, budget, technology and time to implement all identified requirements. Prioritising requirements means the most important needs are delivered first within these constraints.

This implies that scope must be controlled. Changing requirements during development can cause scope creep. Software requirements are often changed by changing business environments, organisational factors and disagreements between stakeholders, which create uncertainty and increase requirements growth, Zowghi and Nurmuliani (2002) explains.

Scope, schedule and cost

Scope affects the amount of work needed - and this affects schedule and cost. Suliman and Kadoda (2017) define software project management as balancing scope, time and cost with proper allocation of resources. They cite poor planning / undefined goals / communication problems and constant scope changes as causes of deadline and budget problems. The empirical evidence for this relationship is provided by requirements volatility: Zowghi and Nurmuliani (2002) observed that requirements volatility affected both schedule overruns and cost overruns.

This means adding or changing requirements cannot be treated as a pure technical decision. A new need may require more development work, more resources, a longer schedule and higher costs. Consequently, cutting the budget or shortening the deadline might also make the project team trim scope or reconsider what requirements can be fulfilled realistically.

Schedule and resources

Schedule and resources also relate. Software development effort partly depends on staff numbers and availability, so trying to shorten a project schedule may require additional staffing and effort. Symons (2012) says software estimators must consider such questions as how much extra effort is needed to deliver a project earlier and how staffing constraints affect duration. Analysing real project data he found a consistent effort/duration trade-off: projects with more staffing were faster but relatively less efficient and projects with less staffing took longer. So, a fixed deadline might require additional staff or effort, while a tight schedule might expand.

However, adding resources does not automatically solve a scheduling problem. Symons (2012) notes that real projects are also affected by management pressures, changing requirements and unforeseen circumstances. Thus, project managers should consider the skills and experience of the team available rather than just number resources. Suliman and Kadoda (2017) also identify staff weaknesses, lack of commitment and turnover of qualified personnel as important influences on software estimation.

Cost and quality

Cost constraints may also impact quality. A project under pressure to stay within budget might have fewer resources available for development, testing or quality assurance. In turn, quality may take longer and cost more money. In addition to quality management, economic management and schedule management, Monserrat, Mas and Mesquida (2025) identify project planning, scope definition, stakeholder management, risk management and change management as project factors. This shows that quality should not be treated as something that is evaluated only after the software has been programmed. It relates to scope, schedule, resources and project management decisions throughout the project.

This is particularly relevant when stakeholder needs are concerned, as technically finishing a project does not necessarily mean that it succeeded.

Stakeholder satisfaction and the achievement of stakeholder expectations are relevant measures of project success, Monserrat, Mas and Mesquida (2025) points out. A product delivered on time and within budget but not satisfied by its users is therefore still unsuccessful.

Risk across all project factors

Risk links the other factors together because uncertainty affects any project objective. Valentino, da Silva and Neves (2020) define risk as "an uncertain condition which may affect project objectives positively or negatively" and note that risks may affect cost, product launch time, financial performance or strategic objectives. Their review also notes that software organisations are exposed to risks of changing requirements and scope in unstable environments. Thus, effective risk management requires finding uncertainty early in the project and monitoring it against the background rather than reacting only when problems occur.

Overall, the relationships can be represented as a chain:

Stakeholder needs → requirements and scope → required effort and resources → schedule and cost → quality → stakeholder satisfaction, with risk influencing every stage.

A shift in stakeholder need can widen scope; increased scope might mean additional resources and effort; further effort can add cost or prolong the schedule; schedule or cost pressure then may compromise quality; and poor quality may ultimately lower stakeholder satisfaction and project success. This explains why software engineering needs project management and decision-making besides programming.

Real-world example: Case C

Ahonen and Savolainen (2010) provide a useful real-world example through their post-mortem analysis of five cancelled software projects. Their **Case C** involved the contractual development of a new tailored system intended to replace an existing one. The planned project duration was 14 months, and the project had a planned effort of 3,570 effective working days. The actual project lasted 15 months and used more than 4,000 effective working days, with a team of 22 people. Although the customer paid for the system, the customer ultimately did not use it.

Decision: During the tendering and negotiation phase, both the supplier's and customer's experts were aware that the proposed schedule was too tight, but management nevertheless decided to accept the risk and sign the agreement. Ahonen and Savolainen (2010) find agreeing to the customer's demand for a tight schedule as the critical mistake in Case C.

Constraint/trade-off: The project was therefore driven by an unrealistic schedule constraint. The lack of flexibility meant that several features were dropped to keep the project on time, while the customer did not have enough time to approve prototypes. This shows the interaction between schedule, scope and quality: protecting the deadline resulted in reductions to scope and insufficient time for stakeholder involvement and validation.

Downstream consequence: Although the project was completed almost on time and almost within budget, the resulting software had poor quality and missing features and was not the software the customer wanted. The customer decided not to use it, meaning that the project effectively failed despite performing relatively well against schedule and budget. Ahonen and Savolainen (2010) conclude that the cancellation of Case C was fundamentally caused by the schedule being too tight.

This case illustrates why software engineering decisions should not isolate one project constraint. Keeping the schedule worked well in the short run, but the trade-off harmed scope, stakeholder involvement and quality resulting in software that was not used.

Question 2: Thinking Ahead - Decisions Across the Software Lifecycle

1. Requirements and scope

Defining requirements and scope is one of the most critical early decisions in the software lifecycle, as it establishes the project boundaries and sets the foundation for all subsequent activities. This decision is brought in at the very beginning of the project and becomes operationally significant throughout its duration. To make a sound decision, project teams must gather information such as user needs, business goals, regulatory requirements, feasibility studies and stakeholder interviews. Clear requirements directly enable design, architecture, testing and project planning, while poorly defined scope often leads to scope creep and costly rework. Evidence from the Project Management Institute's **Pulse of the Profession 2024 Report** highlights this risk, noting that 32% of projects experienced scope creep and 12% failed outright (Pierre Le Manh, 2024), underscoring the importance of addressing requirements and scope early in the lifecycle.

2. Methodology and process

Choosing the right methodology and process is a crucial early decision in the software lifecycle because it aligns the team's workflow and sets the rhythm for project execution. This decision should be considered at project initiation, before development begins and becomes operationally significant during planning and the early development sprints. To make a sound choice, teams must evaluate factors such as team size, project complexity, stakeholder involvement and delivery timelines. The selected methodology directly influences project scheduling, communication, testing approaches and delivery cadence, while poor alignment can lead to delivery friction and inefficiencies. Evidence from the Project Management Institute's **Pulse of the Profession 2024 Report** reinforces this point, showing that projects using agile or adaptive methods achieved a 73% success rate compared to traditional approaches (Pierre Le Manh, 2024), highlighting the importance of methodology selection at the outset.

3. Stakeholder alignment

Stakeholder alignment is a vital early decision in the software lifecycle because it ensures a shared vision and prevents conflicting goals from derailing the project. This consideration begins at the requirements gathering stage and remains operationally significant throughout requirements validation and design approval. To make a sound decision, project teams must understand stakeholder goals, priorities, success metrics and communication preferences. Effective alignment enables requirements validation, design sign-off, acceptance testing and change management, while poor alignment often results in conflicting goals and project delays. As Kerzner's *Project Management Handbook* (Nieto-Rodriguez, 2021) emphasizes, lack of stakeholder alignment and weak sponsorship are major causes of project failure (Nieto-Rodriguez, 2021), highlighting the importance of securing stakeholder consensus early in the lifecycle.

4. Security and compliance

Security and compliance are critical decisions that must be addressed early in the software lifecycle to protect data and maintain user trust. These considerations should be introduced at the system design stage, before coding begins, and remain operationally significant throughout architecture, coding and deployment phases. To make sound decisions, teams must account for applicable regulations such as GDPR or HIPAA, conduct threat modelling, assess data sensitivity and define authentication requirements. Early attention to these factors directly influences database design, API development, testing, deployment and ongoing maintenance. Neglecting security and compliance can result in breaches and costly fines, while proactive planning reduces long-term risks. Evidence from IBM's *Cost of a Data Breach Report 2023* demonstrates that organizations incorporating security design early significantly lower breach costs, reinforcing the importance of embedding compliance and security considerations from the outset (IBM Security, 2023).

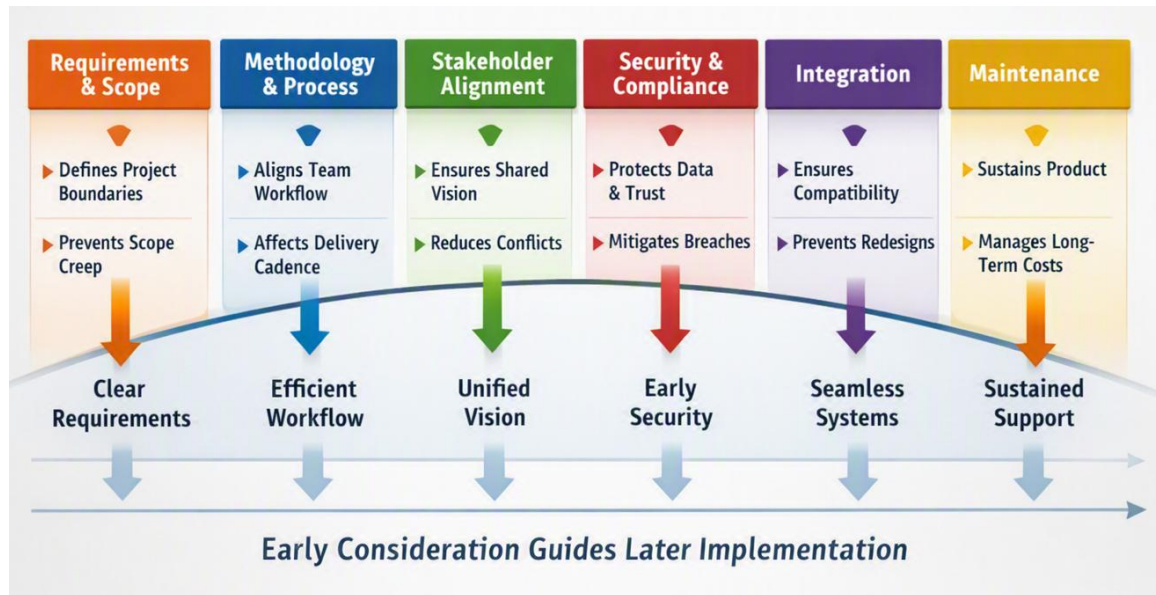
5. Integration

Integration is a critical decision in the software lifecycle because it ensures compatibility between systems and prevents costly redesigns later. This concern should be addressed during system design and architecture planning, and it becomes operationally significant when APIs and interfaces are developed and tested. To make sound integration decisions, teams must consider existing systems, interoperability standards, API specifications and vendor dependencies. These choices directly affect system testing, deployment and long-term scalability, while poor planning often results in redesigns and delays. Gartner reports emphasize that integration challenges are among the top drivers of IT project overruns, underscoring the importance of addressing compatibility and interoperability early in the lifecycle.

6. Maintenance

Maintenance is a fundamental decision in the software lifecycle because it sustains the product and ensures its long-term viability. This concern should be considered as early as the design stage, before implementation and becomes operationally significant post deployment during support and updates. To make sound maintenance decisions, teams must plan versioning strategies, set up documentation standards, distribute support resources and define a long-term roadmap. These choices directly affect bug fixing, patching, upgrades and user support, while neglecting maintenance planning often results in high long-term costs. Evidence from IEEE Software Engineering studies confirms that maintenance consumes between 60–80% of total lifecycle costs (Haleem, 2023), highlighting the importance of embedding maintenance considerations into the design phase to reduce future burdens.

Decision horizon map



Question 3: Evidence-Based Technology Stack Decisions

3.1 Evaluation context and alternatives

This comparison assumes a small team with exposure to web fundamentals but no proven production advantage in either tech stack. The hypothetical system has certain requirements like a browser interface, relational data, and automated testing. These assumptions are evaluation inputs, not CivicConnect requirements.

To isolate the main engineering trade-off, both approaches use a React user interface and PostgreSQL database, the variable is the server-side platform. Approach A uses TypeScript, Node.js and Express. Approach B uses C# and ASP.NET Core on .NET. TypeScript adds static type checking while keeping JavaScript runtime behaviour (TypeScript, n.d.). Express requires application teams to apply the production hardening described in the security guidance, where ASP.NET Core has a more integrated set of security and testing facilities (Express.js, n.d.; Microsoft, 2026a; Microsoft, 2026b).

Layer	Approach A: TypeScript stack	Approach B: .NET stack
Browser interface	React with TypeScript	React with TypeScript
Server/API	Node.js LTS, Express and TypeScript	Supported .NET LTS, ASP.NET Core and C#
Data	PostgreSQL	PostgreSQL
Toolchain	npm, TypeScript compiler and node: test	NuGet, .NET CLI and MS Test/xUnit/NUnit

Using supported release lines matters in both approaches. At the time of research, Node.js 24 was an LTS line and the Node.js project advised production applications to use Active LTS or Maintenance LTS releases. .NET 10 was an active LTS release supported until November 2028 (Node.js, 2026; Microsoft, 2026c). These dates demonstrate that lifecycle policy, not simply the framework name, forms part of a stack decision.

3.2 Comparison against lifecycle criteria

Team capability and learning curve

Approach A can use TypeScript across the browser and API layers; this reduces language switching and can allow types for API data to be shared. TypeScript does not change JavaScript's runtime behaviour so the team must still understand JavaScript and the selected middleware (TypeScript, n.d.). Approach B introduces C# on the server, which increases initial learning for a team without .NET experience. The matrix therefore gives approach A has small advantage under the stated neutral-skills assumption. An actual skills audit could reverse this result.

Security ecosystem

ASP.NET Core has integrated facilities for authentication, authorization, data protection, HTTPS enforcement, secret storage and more (Microsoft, 2026a). Express requires teams to configure transport layer security, distrust and validate user input, add security headers, protect authorization endpoints and audit dependencies (Express.js, n.d.). This flexibility is valuable, but it creates more assembly and configuration decisions that must be managed.

The higher .NET score reflects integrated coverage, not a claim that ASP.NET Core applications are automatically secure.

Maintainability

TypeScript's static checker detects many type-related mistakes before execution that improves feedback during changes (TypeScript, n.d.). The Node.js/Express approach can be modular, but its structure and many conventions must be selected and enforced by the team. ASP.NET Core groups major web concerns and its testing within a coordinated platform (Microsoft, 2026a; Microsoft, 2026b). Approach B therefore scores higher for day-to-day consistency under a small team, while Approach A remains a strong fit when the team enforces shared conventions.

Testing and tooling support

Node.js includes a stable built-in test runner with command-line execution and watch support (Node.js, n.d.). The wider ecosystem can add test libraries, coverage and browser testing but tool selection and configuration must be decided by the team. .NET's has a testing surface covers unit, integration and load testing, with CLI and continuous-integration support and multiple established frameworks including MS Test, NUnit and xUnit (Microsoft, 2026b). Approach B receives the stronger score because the testing platform, command-line workflow and framework integration is seen as one coordinated toolchain.

Deployment and platform compatibility

Both approaches provide supported production-oriented release lines and no platform constraint in the generic scenario disqualifies either candidate (Node.js, 2026; Microsoft, 2026c). They therefore receive the maximum fit score under the stated portability assumption. A real deployment constraint such as an approved operating system, hosting platform, or an architecture standard could act as a hard gate instead of a weighted preference.

Development and likely operational cost

Because both alternatives share React and PostgreSQL, those common components do not differentiate cost. No workload, staffing, hosting or support quotations are available so assigning either alternative a cost advantage would not be possible thus both receive the same score. Reliable operational costing requires workload and deployment evidence that is unavailable in this pre-project exercise.

Future change and maintainability implications

Node.js production guidance recommends LTS releases and describes approximately 30 months of critical-fix coverage, and its release model is changing from the historic six-month major cadence to an annual cadence from Node.js 27 (Node.js, 2026). .NET publishes a major release each November, while even-numbered LTS releases receive three years of support and patches (Microsoft, 2026c). Both require planned upgrades and dependency monitoring. Approach B scores higher because its platform-wide LTS policy covers the runtime, ASP.NET Core and Entity Framework Core, while the Express approach also depends on separately governed npm packages and middleware.

3.3 Criteria weights and justification

The weights represent a generic information-system context and total 100%. Security is highest because failures in authentication, authorization or data handling can directly harm users and invalidate deployment. Capability, maintainability, testing and future change are close behind because they influence delivery risk and total lifecycle effort. Deployment and cost receive lower weights because the scenario supplies neither a differentiating platform constraint nor credible cost measurements.

Criterion	Weight	Why this relative importance is justified
Team capability and learning curve	15%	Skills affect delivery speed, defect risk, and the ability to maintain the result.
Security ecosystem	20%	A web information system must protect identities, access, and data throughout its lifecycle.
Maintainability	15%	Clear boundaries and consistent conventions reduce routine change and support effort.
Testing and tooling support	15%	Automated evidence is needed to detect regression and support-controlled delivery.
Deployment/platform compatibility	10%	Portability matters, but both approaches already satisfy the generic cross-platform assumption.
Development and operational cost	10%	Cost matters, but credible differentiation needs workload, staffing and hosting evidence not yet available.
Future change implications	15%	Support horizons and upgrade obligations affect security, technical debt, and long-term viability.

3.4 Weighted decision matrix

Scores use a 1-5 scale: 1 = weak fit, 2 = limited fit, 3 = adequate fit, 4 = strong fit and 5 = very strong fit under the stated assumptions. Weighted points are calculated as weight x score / 5, producing a maximum total of 100. Scores describe relative contextual fit, not universal technology quality.

Criterion	Weight	TypeScript score / points	.NET score / points
Team capability and learning curve	15%	4 / 12	3 / 9
Security ecosystem	20%	4 / 16	5 / 20
Maintainability	15%	4 / 12	5 / 15
Testing and tooling support	15%	4 / 12	5 / 15
Deployment/platform compatibility	10%	5 / 10	5 / 10
Development and operational cost	10%	4 / 8	4 / 8
Future change implications	15%	4 / 12	5 / 15
TOTAL	100%	82 / 100	92 / 100

Under these assumptions, Approach B scores 92 and Approach A scores 82. The difference is mainly created by ASP.NET Core's integrated security surface, coordinated testing tools and platform-wide LTS policy.

3.5 Limitations, sensitivity and critical conclusion

The weights are judgements and the 1-5 scores are not precise measurements, criteria can overlap, and implementation quality is not captured. Maintainability and future change are separated by the everyday code structure, and release support and upgrade burden. The model also lacks measured workload, team skills, security tests and more.

The result is sensitive to context. For example, if an actual team were highly proficient in TypeScript but novice in C#, changing only the capability scores from 4 versus 3 to 5 versus 1 would move the totals to 85 and 86 respectively. The ten-point baseline gap would almost disappear.

Therefore, the highest-scoring or most popular stack is not automatically correct for every project. A score is only the output of selected criteria, weights, evidence and assumptions. A defensible decision requires validated requirements, hard-constraint checks, team evidence, and small technical experiments where uncertainty is high.

Question 4: Generative AI in Software Engineering

4.1 Useful applications of generative AI in software engineering

Generative AI can support software engineering by performing repetitive/time consuming tasks for developers while delegating important decisions and verification to humans. Coding, software testing and documentation are three of the applications.

Code generation: Generative AI will produce code from natural language instructions, complete existing code and produce repetitive programming structures. This may increase developer productivity by reducing time for routine implementation. Using GitHub Copilot, for example, completed a programming task 55% faster than non-users (Kalliamvakou et al., 2022).

Software testing: Unit tests and test cases can be generated from existing source code and documentation using Generative AI. This may decrease manual effort in building test suites and increase test coverage. Empirical evaluation of the TestPilot system revealed that LLM-generated tests had median statement coverage of 70.2% and branch coverage of 52.8% for 25 JavaScript packages (Schafer et al., 2024).

Documentation: Generative AI may produce software documentation including code explanations, comments, API descriptions and technical summaries. Observations from developers themselves regarding their practical use of GenAI show that documentation is perceived as reliable when compared with more complex tasks like code generation and debugging (Otten et al., 2025).

Overall, such applications are useful to reduce repetitive work and increase developer productivity; But output still needs human review before acceptance into a software project.

4.2 Risks of AI-assisted engineering

1. Bias and reliability of outputs

One significant risk of AI assisted engineering is the presence of bias and reliability issues in its outputs. Because AI systems are trained on historical data, they can inherit biases or reflect incomplete information. This can lead to skewed engineering decisions, such as overlooking critical safety factors or misallocating resources. For example, if the training data underrepresents certain failure scenarios, the AI may fail to predict them, resulting in unsafe designs.

2. Overdependence and reduced human oversight

Another limitation is the danger of over-dependence on AI, which can reduce human oversight and accountability. Engineers may rely too heavily on AI recommendations without critically evaluating them, allowing flawed or unsafe solutions to pass unchecked. In safety critical fields such as aerospace or medical devices, blind reliance on AI outputs could have catastrophic consequences if the system makes an error. Human judgment and validation remain essential to ensure that AI suggestions are sound and trustworthy.

3. Transparency and explainability limitations

A third risk lies in the transparency and explainability of AI systems. Many advanced models, particularly deep learning, operate as “black boxes,” making it difficult to understand how they reach conclusions. This lack of explainability undermines trust, complicates regulatory approval, and makes debugging or auditing engineering decisions challenging. For instance, if an AI proposes a design change but cannot explain its reasoning, engineers may struggle to justify the decision to stakeholders or regulators.

4.3 Human verification and accountability

Human verification and control process

- 1. Classify and contain.** Identify what the output affects like requirements, architecture and source code. Record its intended use and risk level. Keep generated code or commands in an isolated branch or sandbox, and do not expose confidential data or grant the AI unnecessary privileges.
- 2. Trace claims and provenance.** Verify factual claims, APIs, citations, licences and dependencies against authoritative sources. Reject fabricated, unavailable or outdated references. NIST specifically recommends reviewing sources and citations and documenting data or content lineage (NIST, 2024).
- 3. Perform competent human review.** A person with the relevant experience and technical competence should compare the output against approved requirements, architecture, coding standards, privacy obligations and threat models. Material or high-risk changes receive independent peer or specialist review. The reviewer must understand the result rather than approve it because it appears correct.
- 4. Test proportionately.** Run the same or stronger controls applied to human-created work like compilation and static analysis, unit and integration tests, negative and boundary tests and security testing where relevant. Generated text or code must be validated and safely encoded before it reaches interpreters, databases, browsers or operating-system functions. NIST recommends documented pre-deployment testing and evaluation proportionate to the risk (NIST, 2024).
- 5. Approve, record and monitor.** Place accepted output under normal version control and change control. Make sure to record the evidence, tests, corrections and rejected suggestions and also keep the authorised human approval and then monitor the deployed change and ensure there is a rollback path. NIST notes that generative AI can require additional human review, tracking, documentation and management oversight (NIST, 2024).

Accountability

If an engineer accepts AI-generated output that later causes a defect, vulnerability or incorrect decision, accountability remains with the humans and organisation responsible for the product. The AI system is a tool: it does not understand the product context, hold professional duties, approve a release or answer to affected stakeholders. The accepting engineer remains responsible for applying competent judgement, verification and escalation. The reviewer or release authority is accountable for the approval decision, while the organisation is accountable for governance, staffing, controls and remediation. Precise legal liability depends on jurisdiction and contracts, but professional accountability cannot be transferred to the model.

This conclusion follows the IEEE-CS/ACM Software Engineering Code of Ethics. It requires engineers to accept responsibility for their own work and to approve software only when they have a well-founded belief that it is safe, meets specifications and passes proper tests (IEEE-CS/ACM Joint Task Force, n.d.). NIST also treats human oversight roles and responsibilities as information that organisations should explicitly define and document (NIST, 2024). Accountability is therefore shared across responsible human roles, but it is not displaced by AI involvement.

AI Research and Verification Record

Record 1	
Team member	Ryno
AI tool/use	ChatGPT
Purpose	To divide the project into sections/tasks for each team member
Output used?	Yes, as a guide for project division
How independently verified	The team reviewed the suggested division and adjusted it based on the project requirements and each member's responsibilities
What was changed/rejected	Tasks were adjusted where necessary to ensure the workload was fairly distributed
Record 2	
Team member	Ryno, Willem and Steven
AI tool/use	ChatGPT and Copilot
Purpose	Spelling and grammar checking
Output used?	Yes, for correcting spelling and grammar
How independently verified	Each member reviewed the corrections and checked that the wording still accurately represented their own work
What was changed/rejected	Spelling and grammar corrections were accepted where appropriate; no content or ideas were changed

References

- Ahonen, J.J. and Savolainen, P. (2010) 'Software Engineering Projects May Fail Before They Are Started: Post-Mortem Analysis of Five Cancelled Projects', *Journal of Systems and Software*, 83, pp. 2178–2187.
- Hujainah, F., Abu Bakar, R., Abdulgaber, M.A. and Zamli, K.Z. (2018) 'Software Requirements Prioritisation: A Systematic Literature Review on Significance, Stakeholders, Techniques and Challenges', *IEEE Access*, 6, pp. 71497–71523.
- Monserat, M., Mas, A. and Mesquida, A.-L. (2025) 'A Study of Factors That Influence the Software Project Success', *Journal of Software: Evolution and Process*, 37, e2735. doi: 10.1002/smr.2735.
- Suliman, S.M.A. and Kadoda, G. (2017) 'Factors that Influence Software Project Cost and Schedule Estimation', *2017 Sudan Conference on Computer Science and Information Technology (SCCSIT)*.
- Symons, C. (2012) 'Exploring Software Project Effort versus Duration Trade-offs', *IEEE Software*, July/August, pp. 67–74.
- Valério, K.G.O., da Silva, C.E.S. and Neves, S.M. (2020) 'Risk Management in Software Development Projects: Systematic Review of the State of the Art Literature', *International Journal of Open Source Software and Processes*, 11(1). doi: 10.4018/IJOSSP.2020010101.
- Zowghi, D. and Nurmuliani, N. (2002) 'A Study of the Impact of Requirements Volatility on Software Project Performance', *Software Engineering Conference, 2002. Ninth Asia-Pacific*.
- Kalliamvakou, E. et al. (2022) 'Research: Quantifying GitHub Copilot's impact on developer productivity and happiness', GitHub Research.
- Schäfer, M. et al. (2024) 'An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation', *IEEE Transactions on Software Engineering*, 50(1), pp. 85–105.
- Otten, D. et al. (2025) 'Prompting in Practice: Investigating Software Developers' Use of Generative AI Tools'.
- Haleem, M., 2023. *A Critical Review for Software Maintenance Cost Estimation Models: A Data Driven Approach*, s.l.: IEEE.
- IBM Security, 2023. *Cost of a data breach report 2023*, s.l.: IBM Security.
- Nieto-Rodriguez, A., 2021. *Harvard Business Review*. 2021 ed. s.l.:Harvard Business school Publishing Corporation.
- Pierre Le Manh, 2024. *Pulse of the Profession*, s.l.: University of Southampton Institutional Repository.
- Express.js (n.d.) 'Production best practices: security'. Available at: <https://expressjs.com/en/advanced/best-practice-security/> (Accessed: 2 September 2026).
- IEEE-CS/ACM Joint Task Force on Software Engineering Ethics and Professional Practices (n.d.) 'Code of ethics for software engineers'. Available at: <https://www.computer.org/education/code-of-ethics> (Accessed: 2 September 2026).
- Microsoft (2026a) 'ASP.NET Core security topics'. Last updated 19 May 2026. Available at: <https://learn.microsoft.com/en-us/aspnet/core/security/?view=aspnetcore-10.0> (Accessed: 2 September 2026).

Microsoft (2026b) 'Testing in .NET'. Last updated 27 April 2026. Available at: <https://learn.microsoft.com/en-us/dotnet/core/testing/> (Accessed: 2 September 2026).

Microsoft (2026c) '.NET and .NET Core official support policy'. Last updated 11 August 2026. Available at: <https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core> (Accessed: 2 September 2026).

National Institute of Standards and Technology (NIST) (2024) Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1. Available at: <https://doi.org/10.6028/NIST.AI.600-1> (Accessed: 2 September 2026).

Node.js (n.d.) 'Test runner'. Available at: <https://nodejs.org/api/test.html> (Accessed: 2 September 2026).

Node.js (2026) 'Node.js releases'. Available at: <https://nodejs.org/en/about/previous-releases> (Accessed: 2 September 2026).

TypeScript (n.d.) 'TypeScript for the new programmer'. Available at: <https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html> (Accessed: 2 September 2026).